

UI_part3

<https://gemini.google.com/app/a3d824fbd104a6a1>

User prompt: Teach me how einsum works and how to use the einops library with NumPy (import numpy as np; from einops import einsum, rearrange), using visual, interactive mini-games. Keep the session ~15–20 minutes. Sections (one mini-game each) 1) The core rule: named axes; an axis missing from the output gets summed 2) rearrange: transpose and reshape ("h w -> w h", "(h w) -> h w") 3) Sums with einsum: total sum, sum over rows vs columns 4) Dot product & outer product ("i, i ->" vs "i, j -> i j") 5) Matrix multiplication ("i j, j k -> i k") and how it relates to @ 6) Batched matrix multiplication ("b i j, b j k -> b i k") Mini-games - For each section, give a short explanation, then an interactive visual widget (not just text). For example: arrays as colored grids with labeled axes, predict the output shape of a pattern, highlight which elements get multiplied and summed, watch an axis disappear (summed) or move (rearranged), or build a pattern string for a given operation. - Keep each widget small and focused on one idea, with instant visual feedback (correct parts light up, wrong ones are marked). - My answers must also appear in the chat (e.g. the widget sends my answer as a message), so the transcript shows our interaction. Guidelines - Use einops syntax: einsum(a, b, "i j, j k -> i k") with space-separated axis names and the pattern as the last argument. Mention np.einsum only briefly for comparison. - Adjust difficulty: if I miss 2 in a row -> slow down + simpler; if I get 2 quickly -> a bit harder or add a twist. - Tiny, deterministic examples (small numbers, shapes like (2,3)). Show intermediate steps. - Code: ≤2 blocks per message, each ≤20 lines, only the imports above. - Do NOT solve graded work. If I paste any, refuse and give a similar practice item. - Be CONCISE. Focus on intuition, simple notation, short variable names. - I may interrupt and ask to start somewhere else. Adapt from there. - One step at a time. Wait for my answer before continuing. - Give a hint first; reveal answers only after I try or ask. Flow 1) Core idea (≤6 sentences): intuition + minimal notation + why it matters + one common pitfall, with one visual. 2) Quick check-in: 2 short questions about my prior exposure and goal. 3) Sections 1–6, each with its mini-game. 4) 3 final quick checks of increasing difficulty, combining the sections. 5) Knowledge check: interactive flashcards and a short quiz (5–8 items) covering all sections, e.g. pattern -> output shape, operation -> pattern, and "what gets summed?". Focus more on topics I got wrong earlier. Show my score at the end. 6) Assessment: what I understood well, what needs work, what to review. 7) Session reflection: ask me for feedback on the session (what worked, what to improve, pacing/difficulty). 8) Recap: a 60-second summary of the key ideas. Start with step 1.

Response: einops makes tensor manipulation readable by treating dimensions as named variables rather than obscure numerical indices. The core rule of einsum (Einstein summation) is beautifully simple: **any axis present in the input but missing in the output gets summed out**. This replaces messy nested for loops with concise string patterns like "i j -> i". It matters because it makes your code self-documenting and eliminates entire classes of shape mismatch bugs. One common pitfall: forgetting the spaces between axis names in einops—always write "i j", not "ij".

Here is a quick visual to ground our mental model of named axes:

Before we dive into the first mini-game, let's do a quick check-in so I can tailor the difficulty:

1. How much prior experience do you have with standard NumPy or PyTorch operations (like `.transpose()`, `.reshape()`, or `np.dot()`)?
2. What is your main goal for learning einops today (e.g., deep learning models, data wrangling)?

(Let me know your answers, and we'll start Section 1!)

User prompt: I've worked with NumPy/PyTorch operations before, so I have some basic experience with them. I mainly want to review and reinforce my knowledge of how einops works.

Response: Awesome, having prior NumPy/PyTorch experience will make this smooth and intuitive!

Section 1: The Core Rule (Named Axes & Disappearing Dimensions)

In `einops.einsum`, every axis gets a name (like `i`, `j`, or full words like `row`, `col`).

The universal rule:

Input axes → **Output axes**

If an axis is on the left side of `->` but **missing** on the right side, it is **summed over** (contracted).

```
import numpy as np
from einops import einsum

x = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape: (2, 3) -> (i, j)

# Keep 'i', omit 'j': sums across the 'j' axis (columns)
row_sums = einsum(x, "i j -> i") # output shape: (2,), values: [6, 15]
```

Mini-Game 1: The Disappearing Axis

Try the interactive puzzle below: identify which axis disappears and predict the resulting shape and values.

Your Turn:

If we run:

```
einsum(x, "i j -> j")
```

1. Which axis gets summed out (`i` or `j`)?
2. What is the output shape, and what are the exact resulting numbers?

User prompt: The `i` axis gets summed out because it is missing from the output. The output shape is `(3,)`. For example, if `x = [[1, 2, 3], [4, 5, 6]]`, the result is `[5, 7, 9]`

Response: Spot on! `i` disappears from the output pattern, so rows are collapsed, leaving shape `(3,)` with values `[1+4, 2+5, 3+6] = [5, 7, 9]`.

Section 2: rearrange (Transpose, Reshape & Flattening)

While `einsum` does math (multiplication and summation), `rearrange` is for moving, reshaping, or flattening dimensions without changing any values.

Key rules:

1. **Transpose / Permute:** Reorder the names.

"`h w -> w h`" swaps rows and columns (matrix transpose).

2. **Flatten / Composition:** Parentheses group or split dimensions.

"(h w) -> h w" splits a 1D vector into a 2D matrix.

"h w -> (h w)" flattens a 2D matrix into a 1D vector.

```
import numpy as np
from einops import rearrange

x = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2, 3)

transposed = rearrange(x, "h w -> w h") # shape: (3, 2)
flattened = rearrange(x, "h w -> (h w)") # shape: (6,) -> [1, 2, 3, 4, 5, 6]
unflattened = rearrange(flattened, "(h w) -> h w", h=2) # back to (2, 3)
```

Mini-Game 2: The Rearrange Shuffle

Test how dimensions reorder and group in the interactive widget below:

Your Turn:

Given an image tensor `img` of shape (batch, channels, height, width):

1. What pattern string would you give `rearrange` to swap it into channels-last format: (batch, height, width, channels)?
2. If you want to flatten just the spatial dimensions `height` and `width` into a single sequence dimension `seq` while keeping `batch` and `channels` untouched, how would you write the pattern?

User prompt: `rearrange(img, "b c h w -> b h w c")` `rearrange(img, "b c h w -> b c (h w)")`

Response: Both are 100% correct! You've got `rearrange` down cleanly.

Section 3: Sums with `einsum` (Total Sum vs. Directional Reductions)

Now let's revisit `einsum` reductions. When you want to reduce dimensions:

1. Total Sum (everything reduced to a scalar):

Omit all axes on the right side of the arrow.

"i j -> " → returns a 0D scalar.

2. Column Sum (collapse rows):

Omit i: "i j -> j" → shape (w,).

3. Row Sum (collapse columns):

Omit j: "i j -> i" → shape (h,).

```
import numpy as np
from einops import einsum

x = np.array([[1, 2, 3],
```

```

[4, 5, 6]]) # shape (2, 3)

total = einsum(x, "i j -> ") # 21 (scalar)
col_sum = einsum(x, "i j -> j") # [5, 7, 9] (sums down rows)
row_sum = einsum(x, "i j -> i") # [6, 15] (sums across columns)

```

Mini-Game 3: The Summation Filter

Your Turn (Quick twist):

Given a 3D tensor x with shape (batch, time, feat):

1. Write the pattern to compute the **mean or sum across the time dimension** only, preserving batch and feat.
2. What pattern produces a single scalar total sum across all elements in x ?

User prompt: For the sum across time: einsum(x, "b t f -> b f") For the mean, einsum itself doesn't divide by the number of elements, so I'd use: einsum(x, "b t f -> b f") / x.shape[1] For a single scalar total sum: einsum(x, "b t f -> ")

Response: Spot on, and great call noting that einsum only sums, so dividing by $x.shape[1]$ handles the mean!

Section 4: Dot Product & Outer Product

When multiplying two 1D vectors u and v (each of length N):

1. **Dot product:** Multiply elementwise and sum everything into a scalar.
 - Both share the axis i , and i is omitted in output:
 - Pattern: "i, i -> "
2. **Outer product:** Multiply every element of u with every element of v to create an $N \times M$ matrix.
 - Input axes have different names (i and j), and both remain in the output:
 - Pattern: "i, j -> i j"

```

import numpy as np
from einops import einsum

```

```

u = np.array([1, 2, 3])
v = np.array([4, 5, 6])

```

```

dot_prod = einsum(u, v, "i, i -> ") # 1*4 + 2*5 + 3*6 = 32 (scalar)
outer_prod = einsum(u, v, "i, j -> i j") # (3, 3) matrix

```

Mini-Game 4: Contraction vs. Expansion

Your Turn:

1. If you have two vectors u and v of the same length and run:

```
einsum(u, v, "i, i -> i")
```

What standard mathematical operation is this (e.g. dot product, outer product, elementwise product), and what is the output shape?

2. If u has length 2 and v has length 3, what happens if you try to run " $i, i \rightarrow$ "?

User prompt: This is an elementwise product, not a dot product. The output shape is $(i,)$, so it has the same length as the input vectors. It will fail because the two i axes must have the same length, but u has length 2 and v has length 3. For a dot product, " $i, i \rightarrow$ " requires matching lengths.

Response: Spot on on both counts! Reusing the label i on both inputs enforces identical lengths and pairs them element-by-element. Keeping i in the output gives elementwise product; dropping it gives the dot product.

Section 5: Matrix Multiplication & Connection to @

Standard matrix multiplication is dot products over rows of A and columns of B :

$$C_{i,k} = \sum_j A_{i,j} \cdot B_{j,k}$$

In `einops.einsum`:

- Matrix A has axes " $i\ j$ " (shape $M \times K$).
- Matrix B has axes " $j\ k$ " (shape $K \times N$).
- Shared axis j matches the inner dimensions and is missing from the output → **it gets summed out**.
- Output axes are " $i\ k$ " (shape $M \times N$).

```
import numpy as np
from einops import einsum
```

```
A = np.array([[1, 2],
              [3, 4]]) # shape (2, 2) -> (i, j)
B = np.array([[5, 6],
              [7, 8]]) # shape (2, 2) -> (j, k)
```

```
# These two are completely equivalent:
C_einsum = einsum(A, B, "i j, j k -> i k")
C_matmul = A @ B
```

Mini-Game 5: The Matrix Multiplication Matchmaker

Your Turn:

1. If A is shape $(2, 3)$ and B is shape $(2, 3)$, can you do $A @ B$ directly? How could you compute $A \cdot B^T$ using a single `einsum` pattern without transposing B first?

2. What operation does `einsum(A, B, "i j, i j -> ")` perform?

User prompt: No, $A @ B$ doesn't work directly because the inner dimensions don't match ($3 \neq 2$). We can compute $A \cdot B^T$ with: `einsum(A, B, "i j, k j -> i k")`.... It performs an elementwise multiplication followed by a sum over all elements so it IS computes the total sum of products (the dot product of the flattened arrays), returning a scalar.

Response: Exactly right on both counts. That flattened dot product is also known as the Frobenius inner product $\langle A, B \rangle_F = \text{Tr}(A^T B)$.

Section 6: Batched Matrix Multiplication (BMM)

Batched matrix multiplication is where `einsum` really outshines traditional tensor syntax. In standard NumPy/PyTorch, handling batch dimensions across multidimensional tensors often requires reshaping, broadcasting tricks, or `.bmm()`.

In `einsum`, you just give the batch dimension a name (e.g. `b`) and include it on both inputs and the output:

```
import numpy as np
from einops import einsum

# Batch of 4 matrices, each of shape (2, 3) and (3, 5)
A = np.random.randn(4, 2, 3) # b i j
B = np.random.randn(4, 3, 5) # b j k

# Batched matmul: batch 'b' preserved, inner 'j' contracted
C = einsum(A, B, "b i j, b j k -> b i k") # shape: (4, 2, 5)
```

Because `b` is on both inputs and in the output, it acts as an **independent slice**—no summing happens across `b`.

Mini-Game 6: The Batch Slicer

Your Turn:

In attention mechanisms (like Transformers), you have:

- Queries Q of shape (batch, heads, seq_q, dim)
- Keys K of shape (batch, heads, seq_k, dim)

1. What `einsum` pattern computes the attention score matrix of shape (batch, heads, seq_q, seq_k)?
 2. What axis was contracted?
-

User prompt: 1. It is `einsum(Q, K, "b h q d, b h k d -> b h q k")` 2. The ``d`` (dim) axis was contracted, because it appears in both inputs but not in the output ...

Response: Nailed it! That is literally the core operation of multi-head self-attention, and you expressed it in one clean line.

Step 4: Final Quick Checks (Increasing Difficulty)

Let's test combining `rearrange` and `einsum` across three progressive puzzles:

1. Warm-up:

Given a square matrix M of shape (n, n) . Write an `einsum` pattern to extract the **trace** (sum of the diagonal elements).

2. Intermediate:

You have image patches P of shape $(\text{batch}, \text{patches}, \text{patch_h}, \text{patch_w}, \text{channels})$ where $\text{patch_h} = 16$, $\text{patch_w} = 16$. You want to un-patchify it into a full image $(\text{batch}, \text{channels}, \text{height}, \text{width})$ where $\text{height} = \text{grid_h} * 16$ and $\text{width} = \text{grid_w} * 16$.

How would you write the `rearrange` pattern using decomposed patch grid dimensions (gh, gw) ?

3. Advanced:

You have a batch of vectors X of shape (b, n, d) and weights W of shape (d, out) . You want to compute XW , but you also want to sum across the sequence dimension n in the exact same step, yielding shape (b, out) . What is the single `einsum` pattern?

Give all three a shot!

User prompt: 1. `einsum(M, "i i ->")` — this should sum the diagonal elements, giving a scalar. 2. `rearrange(P, "b (gh gw) ph pw c -> b c (gh ph) (gw pw)", ph=16, pw=16)` ... 3. `einsum(X, W, "b n d, d o -> b o")` — this multiplies over `d` and also sums over the sequence dimension `n` i think so

Response: You scored **3 out of 3** on the progressive checks!

- Question 1: Reusing `i` on the same matrix indexes the diagonal, and omitting it sums them up (trace).
- Question 2: Perfect Vision Transformer (ViT) patch reconstruction pattern.
- Question 3: Both `d` (feature projection) and `n` (sequence pooling) are omitted from the output, so both contract simultaneously.

Step 5: Knowledge Check (Interactive Quiz & Flashcards)

Let's do a fast 6-question interactive quiz across all sections to lock everything in and compute your final score.

To record your answers in the chat transcript as well, reply with your answers (A, B, C, or D) for these quick 6:

1. **Rule:** If an axis is in the input pattern but absent in the output pattern, what happens?

[A] It is dropped/truncated [B] It is summed over [C] It raises an `IndexError` [D] It is broadcasted

2. **Shape:** If A has shape $(10, 20, 30)$ and we run `einsum(A, "x y z -> z x")`, what is the output shape?

[A] (20, 10) [B] (30, 20) [C] (30, 10) [D] (10, 30)

3. **Rearrange:** Which pattern splits a single dimension of size 12 into a 3×4 grid?

[A] "h w -> (h w)" [B] "(h w) -> h w" [C] "h -> w" [D] "h w -> w h"

4. **Reduction:** What is the output of `einsum(A, "i j -> ")`?

[A] A 1D vector [B] The diagonal elements [C] A 0D scalar (total sum) [D] The transpose of A

5. **Inner vs Outer:** What pattern produces an outer product between 1D vectors u and v ?

[A] "i, i -> " [B] "i, j -> i j" [C] "i, i -> i" [D] "i j, j k -> i k"

6. **Contraction:** In `einsum(A, B, "b i k, b k j -> b i j")`, which dimension is contracted?

[A] b [B] i [C] j [D] k

User prompt: B A B C B D

Response: Here is your score breakdown:

- **Question 1: B** — Correct! Missing axes get summed over.
- **Question 2: A** — *Close miss!* In "x y z -> z x", z corresponds to the 3rd dimension (size 30) and x to the 1st (size 10), so the resulting shape is (30, 10) ([C]). (20, 10) used y instead of z.
- **Question 3: B** — Correct! "(h w) -> h w" un-flattens a dimension.
- **Question 4: C** — Correct! Dropping all output axes sums the matrix into a scalar.
- **Question 5: B** — Correct! Separate axes i, j preserved in output create an outer product matrix.
- **Question 6: D** — Correct! The inner index k is omitted in the output, so it contracts.

Score: 5 / 6 (83%) — Strong performance across all the core ideas.

Step 6: Assessment

- **What you mastered thoroughly:**
 - **Contraction mechanics:** You consistently recognized which axes vanish and get summed out, including complex operations like batched attention scores and joint sequence pooling.
 - **rearrange syntax:** You quickly grasped parenthetical composition/decomposition (like patch flattening and reconstruction for ViT).
 - **Elementwise vs. reductions:** You immediately identified length constraints, elementwise operations ("i, i -> i"), and Frobenius inner products.
- **What to watch out for:**

- **Axis alignment / mapping:** In Q2, mixing up positions (y vs z) is the easiest mistake when patterns permute multiple dimensions. Writing out the mapping x: 10, y: 20, z: 30 before rearranging prevents shape mismatches.

Step 7: Session Reflection

We hit our ~15–20 minute window. How was the pace, difficulty, and format of the mini-games? Was there anything you felt worked particularly well, or anything you'd like to adjust for next time?

Step 8: 60-Second Recap

1. **The Contract Rule:** Any axis written on the input side but missing from the output side is summed over.
 2. **einops.rearrange:** Pure layout manipulation. Reorder names to transpose ("h w -> w h"), or use parentheses to compose/decompose shapes ("h w -> (h w)" vs "(h w) -> h w").
 3. **Repeated Labels = Alignment:** Matching index names across tensors ("i, i") enforces equal dimension size and pairs them element-by-element.
 4. **Preserved Labels = Batch / Slice:** Any dimension repeated in both inputs and the output ("b . . .") is processed independently without summing across it.
-

User prompt: Thank you, that was all

Response: You're very welcome! You did a fantastic job working through those tensor patterns.

Whenever you're ready to explore another topic or build something new, just give me a shout. Happy coding!
